



LLM (大規模言語モデル) 初心者向けガイド

Python開発経験者向け | 知識ゼロからのスタート

目次

1. [従来プログラミング vs AI の違い](#) 2. [LLM の基本](#) 3. [ニューラルネットワークの仕組み](#) 4. [Transformer アーキテクチャ](#) 5. [このプロジェクトで学べること](#)

従来プログラミング vs AI の違い

従来的なプログラミング

入力 → ルール (if/else) → 出力 ↑ 人間が明示的に書く

例：計算機プログラム

```
python def add(a, b): return a + b # ルールが明示的
```

```
print(add(5, 3)) # 出力: 8
```

AI (機械学習) 的アプローチ

入力 → パラメータ (重み) → 出力 ↑ データから学習

違い：ルールが隠れている → データから学習

```
python
```

大量のデータを見て「パターン」を学習

実際のルールは人間には見えない

```
model = LargeLanguageModel() output = model.predict("質問をしてください") # 学習したパターンから出力
```

イメージ図

```
mermaid graph LR
  A["従来プログラミング"] -->|ルールを人間が書く| B["ロボット"]
  B -->|ルール実行| C["予測可能な出力"]
  D["機械学習"] -->|大量のデータ| E["AI"]
  E -->|パターンを学習| F["柔軟な出力"]
  style A fill:#ffcccc style C fill:#ffcccc style D fill:#ccccff style F fill:#ccccff
```

LLM の基本

LLM とは？

LLM = Large Language Model (大規模言語モデル)

- **Large (大規模)** : 数十億個のパラメータ (重み) を持つ - **Language (言語)** : テキストを理解・生成 - **Model (モデル)** : 確率的に次の単語を予測

簡単な例：次の単語予測

「こんにちは、」 → AI が予測 → 「元気ですか？」 「The quick brown」 → AI が予測 → 「fox」

↑ このパターンを数十億個学習

LLM の動作フロー

mermaid sequenceDiagram participant User as ユーザー participant Input as 入力処理 participant Model as LLM モデル participant Output as 出力処理 participant Display as 表示 User->>Input: テキストを入力 Input->>Model: トークン化

(単語に分割) Input->>Model: 数値に変換して送信 Model->>Model: 確率計算

(次の単語を予測) Model->>Output: 予測結果

(確率分布) Output->>Output: トークンをテキストに変換

Output->>Display: 最終的な

回答テキスト Display->>User: 画面に表示

ニューラルネットワークの仕組み

1. 基本ユニット：ニューロン

入力1 — 入力2 — × 重み → 足し算 → 活性化関数 → 出力 入力3 —

イメージ：投票のようなもの python

簡単な例

```
def neuron(inputs, weights, bias): z = sum(i * w for i, w in zip(inputs, weights)) + bias return activation_function(z) # シグモイド関数など
```

例：入力3個、重み3個

```
output = neuron([0.5, -0.3, 0.8], [2.0, -1.5, 0.4], 0.1)
```

2. ネットワーク構造

```
` mermaid graph LR
  A1["入力層  
テキスト"] --> B1["隠れ層1  
パターン検出"]
  A1 --> B2["隠れ層1"]
  A1 --> B3["隠れ層1"]
  B1 --> C1["隠れ層2  
概念把握"]
  B2 --> C1
  B3 --> C1
  B1 --> C2["隠れ層2"]
  B2 --> C2
  B3 --> C2
  C1 --> D["出力層  
次の単語"]
  C2 --> D
  style A1 fill:#ffffcc
  style B1 fill:#ccffcc
  style B2 fill:#ccffcc
  style B3 fill:#ccffcc
  style C1 fill:#ccccff
  style C2 fill:#ccccff
  style D fill:#ffcccc`
```

Transformer アーキテクチャ

LLM の核：Transformer とは

Transformerはサッカーの試合に例えると：

従来のRNN: ボールを順番に回す（時間がかかる） Transformer: 全選手が同時に相手を見る（並列処理）

3つの主要な仕組み

1 Tokenization（トークン化）

```
` mermaid graph LR
  A["こんにちは、  
元気ですか？"] -->|分割| B["こんにちは"]
  A -->|分割| C[" "]
  A -->|分割| D["元気"]
  A -->|分割| E["です"]
  A -->|分割| F["か"]
  A -->|分割| G["？"]
  B -->|数値化| B2["2045"]
  C -->|数値化| C2["101"]
  D -->|数値化| D2["3456"]
  E -->|数値化| E2["1892"]
  F -->|数値化| F2["405"]
  G -->|数値化| G2["51"]
  style B2 fill:#ffcccc
  style C2 fill:#ffcccc`
```

```
` python`
```

Python 例

```
from transformers import AutoTokenizer
```

```
tokenizer = AutoTokenizer.from_pretrained("bert-base-japanese") text = "こんにちは、元気ですか？" tokens = tokenizer.encode(text) print(tokens) # [2, 2045, 101, 3456, 1892, 405, 51, 3]
```

2 Attention（注意機構）

Q: 「何に注目すべき？」 K: 「このトークンに注目してください」 V: 「注目したら、この情報を使います」

例：「銀行」という単語の意味を理解する

文1: 「川の銀行で遊んだ」 ← 「川」 に注目 文2: 「銀行に お金を預けた」 ← 「お金」 に注目

各文脈で異なる意味になる → **Attention が文脈を判断**

```
` mermaid graph TB
  Q["Query  
『銀行』の  
意味は？"] -->|質問投げかけ| K1["Key1: 川"]
  Q -->|質問投げかけ| K2["Key2: お金"]
  Q -->|質問投げかけ| K3["Key3: 預ける"]
  K1 -->|スコア計算| Attention1["Attention  
スコア計算"]
  K2 -->|スコア計算| Attention2["Attention  
スコア計算"]
  K3 -->|スコア計算| Attention3["Attention  
スコア計算"]
  Attention1 --> Output["『金融機"]`
```

関』の
意味で理解"] style Q fill:#ffffcc style Attention fill:#ccffff style Output fill:#ccffcc`

3 Feed-Forward Network（順伝播）

` 隠れ層での非線形変換 密集 → 活性化 → 密集 4倍拡大 削減`
` python`

Transformer の基本構成

```
class TransformerBlock:
    def forward(self, x):
        # Multi-Head Attention
        x = self.attention(x)
        # Feed-Forward
        x = self.feed_forward(x)
        return x
```

🎓 LLM の学習方法

3つの学習フェーズ

` mermaid graph LR
 A["フェーズ1: 事前学習
Pretraining"] -->|何十億個| B["テキストデータ
から学習"]
 B -->|得られるもの| C["一般的な
言語理解"]
 C --> D["フェーズ2: ファインチューニング
Fine-tuning"]
 D -->|特定の| E["タスク用データ
で調整"]
 E -->|得られるもの| F["特定の
タスク対応"]
 F --> G["フェーズ3: 人間フィードバック
RLHF"]
 G -->|人間の| H["評価データで
最適化"]
 H -->|得られるもの| I["人間らしい
高品質な出力"]
 style A fill:#ffcccc
 style C fill:#ffcccc
 style D fill:#ffffcc
 style F fill:#ffffcc
 style G fill:#ccffff
 style I fill:#ccffff`

📈 LLM の大きさと能力の関係

` モデルサイズ 能力 ↑↑ | scaling law | / 100B 現在 / / 10B / / / / +----->`

重要なポイント： パラメータ数が多い = より複雑なパターンを学習できる

` 1B（10億） パラメータ → 単純な文法 10B（100億） → 一般的な質問応答 100B（1000億） → 複雑な推理`

🔍 このプロジェクトで学べること

本プロジェクトは、これらの概念を **実装レベル** で学べます：

| 技術 | 学ぶ内容 | ファイル | |-----|-----|-----| | **Tokenization** | テキスト→トークン変換 | `autonomousragagent.py` || **Embedding** | トークン→数値ベクトル | `embeddings/` || **Fine-tuning** | モデルの特定

理解度チェック

- [] 従来プログラミングとAIの違いが説明できる - [] LLM が「次の単語予測」の繰り返しだとわかった - [] ニューロンと層のイメージが湧いた - [] Attention 機構が文脈判断に使われると理解した - [] Transformer の3つの部分が説明できる - [] 学習フェーズ（事前学習→ファインチューニング→RLHF）の流れが理解できた

次のステップ

 このガイドを読み終わったら → [段階2：プロジェクト全体像](#) へ

次のガイドでは、このプロジェクトの **全体アーキテクチャ** を図解します。

質問やフィードバック: Issue を作成するか、ドキュメント管理者に連絡してください